# Assignment 04

## CAN Driver Module Analysis and Configuration

| | |
|---|---|
| **Level** | Fresher Embedded / MCU Fundamentals |
| **Suggested duration** | 300 minutes |
| **Submission** | Report, source project, diagrams, and test evidence as applicable |
| **Quality gate** | Clear explanation, correct configuration, clean build, and reproducible result |

## 1. Objective

Apply CAN protocol and controller knowledge to design a generic CAN Driver configuration, message-buffer mapping, interrupt flow, error strategy, and nominal bit timing.

## 2. Part A - CAN foundation

- Draw a CAN network with at least three nodes, CAN controllers, transceivers, twisted pair, and termination.
- Draw the generic CAN controller architecture: CPU interface, protocol engine, bit timing, acceptance filters, message buffers, interrupt logic, and error counters.
- Compare standard 11-bit and extended 29-bit data frames. Identify SOF, arbitration, control, data, CRC, ACK, EOF, IDE, RTR, and DLC.
- Explain non-destructive arbitration using two example identifiers.

## 3. Part B - Message buffers and interrupts

- Define two transmit hardware objects and two receive hardware objects. Assign identifier type, ID or filter, DLC, polling/interrupt mode, and software handle.
- Write a transmit sequence from API request through free-buffer check, buffer loading, transmit request, arbitration, completion, and confirmation.
- Write a receive sequence from frame detection through filtering, buffer storage, receive interrupt, upper-layer indication, and buffer release.
- Define expected behavior for busy transmit objects and receive overrun.
- Create ISR pseudocode for Rx, Tx complete, error warning, wake-up, and bus-off sources.

## 4. Part C - Error management

- Explain bit, stuff, CRC, form, and acknowledgement errors.
- Describe the purpose of transmit and receive error counters.
- Describe Error Active, Error Passive, and Bus-Off states.
- Define driver actions and upper-layer notifications for error warning and bus-off.
- Propose a bus-off recovery strategy as a project decision, clearly separating protocol behavior from software policy.

## 5. Part D - Bit timing

- Use a stated CAN controller clock and target bit rate.
- Select prescaler and total time quanta per bit.
- Assign Sync, Propagation, Phase Segment 1, and Phase Segment 2.
- Calculate nominal bit rate and sample point.
- Check that every value fits the selected controller register limits.

```
Nominal bit rate = CAN clock / (Prescaler × Total time quanta)
Sample point (%) = (Sync + Prop + Phase1) / Total time quanta × 100
```

## 6. Minimum configuration scenario

| | |
|---|---|
| **Nominal bit rate** | 500 kbit/s |
| **Transmit objects** | 2 |
| **Receive objects** | 2 |
| **ID coverage** | At least one standard and one extended ID |
| **Interrupts** | Rx, Tx complete, error, and bus-off |
| **Evidence** | Buffer table, timing calculation, flow diagrams, and ISR pseudocode |

## 7. Acceptance criteria

- Frame fields, arbitration, error states, and timing calculations are internally consistent.
- Hardware-specific register values are used only when a target controller manual is explicitly selected.
- Message-buffer ownership, busy, overrun, and interrupt-flag clearing are clearly defined.

## 8. Coding

- S32K144 <-> Tool CAN
- Every 1s, S32K144 transmit value 0 and increase 1.
- Baud-rate: 500 kbit/s
- CAN ID: 0x750
- Can Receive Interrupt
- Tool CAN:
- CAN ID 0x751:
- Byte 4: 1 -> S32K144 continue transmit
- Byte 4: 0 -> S32K144 stop transmit

## Evaluation criteria

| Category | Weight | Expected evidence |
|---|---|---|
| Technical understanding | 30% | Accurate concepts, diagrams, and terminology. |
| Implementation / analysis | 35% | Required configuration, code, or mapping is complete. |
| Testing and evidence | 20% | Results are demonstrated and reproducible. |
| Documentation quality | 15% | Clear structure, naming, references, and conclusions. |